

SVKM's
NARSEE MONJEE COLLEGE OF COMMERCE AND ECONOMICS
EMPOWERED AUTONOMOUS

CIS + VCG MINI PROJECT

A Project Submitted to
Department of Information Technology

By

Name	Roll No	SAP ID
Trupti Naik	A025	45618240055
Nidhi Shah	A059	45618240063

Under the Guidance of
Prof. Ruta Prabhu

Date: 24^h September 2026

Project Topic:

Handwritten Gurmukhi Character Recognition Using Random Forest

Develop a Python-based Random Forest classifier for handwritten Gurmukhi character recognition using a publicly available labeled Gurmukhi character dataset. Students should explore the dataset, preprocess the character images, generate suitable feature representations and train the Random Forest model. The system should predict the class of unseen character images and evaluate its performance using accuracy, precision, recall, F1-score and confusion matrix. The project should include appropriate visualizations of the dataset and predictions and should be maintained using GitHub.

ABSTRACT

This project develops a Python-based Random Forest classifier for handwritten Gurmukhi character recognition. A publicly available dataset containing 41 character classes and 12,128 handwritten images is used. The images are pre-processed and represented using raw pixel features and HOG features. Random Forest models are trained and compared, followed by hyperparameter tuning. The final model achieves 96.54% accuracy, with 96.65% precision, 96.54% recall and 96.53% F1-score. The system also predicts unseen character images and provides visual analysis through classification reports, confusion matrices and prediction visualizations.

Introduction

Handwritten character recognition is an important application of Machine Learning and Pattern Recognition, where a computer is trained to identify characters from handwritten images. Unlike printed text, handwritten characters can vary significantly in their shape, size, stroke thickness, orientation and writing style, making accurate recognition a challenging task.

This project focuses on the recognition of handwritten Gurmukhi characters, the script primarily used for writing Punjabi. A publicly available labelled Gurmukhi character dataset is used to train and evaluate the classification system. The character images are first explored and pre-processed to ensure that they are in a suitable form for machine learning.

The project implements a Random Forest classifier for multi-class character recognition. Different feature representations are explored, including raw pixel features and Histogram of Oriented Gradients (HOG) features, to study their effectiveness for handwritten character classification. The Random Forest model is further optimized through hyperparameter tuning to improve its performance.

The trained model is evaluated on previously unseen test images using accuracy, precision, recall, F1-score and confusion matrix. Visualizations are also used to understand the dataset, model performance and prediction results. The final system demonstrates how traditional machine learning techniques can be applied to recognize handwritten Gurmukhi characters effectively.

AIM

To develop a Random Forest-based machine learning system for recognizing handwritten Gurmukhi characters from image data.

OBJECTIVES

- Explore a publicly available Gurmukhi character dataset.
- Preprocess the handwritten character images.
- Generate suitable image feature representations.
- Train a Random Forest classifier.
- Compare raw pixel and HOG-based representations.
- Optimize the Random Forest model.
- Evaluate the model using accuracy, precision, recall, F1-score and confusion matrix.
- Predict unseen Gurmukhi character images.

SCOPE OF THE PROJECT

The project focuses on character-level recognition of handwritten Gurmukhi images. It covers dataset exploration, image preprocessing, feature generation, Random Forest classification, model evaluation and prediction of unseen character images. The current system is limited to the 41 character classes present in the selected dataset and does not recognize complete words or sentences.

DATASET DESCRIPTION

The project uses a publicly available labelled dataset of handwritten Gurmukhi characters. The dataset contains individual handwritten character images organized into separate classes, with each class representing a distinct Gurmukhi character. In total, the dataset consists of **12,128 images belonging to 41 classes**.

The original character images have a resolution of **256 × 256 pixels**. During preprocessing, the images were converted to grayscale, resized to **32 × 32 pixels**, and normalized to pixel values between **0 and 1**. The processed images were then converted into numerical feature representations suitable for training the Random Forest classifier.

The dataset was divided into training and testing sets using a **stratified 80:20 split**, resulting in **9,702 training images and 2,426 testing images**. The stratified split ensures that the distribution of the 41 character classes is maintained across both sets, allowing the model to be evaluated fairly on unseen handwritten characters.

Link to the Dataset: [Gurumukhi Dataset](#)

Link to Colab Notebook: [Colab Notebook](#)

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Dataset Setup & Exploration

```
[14]
✓ 0s
import os
import zipfile
import shutil
import random

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from PIL import Image

RANDOM_STATE = 42

random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)

print("Libraries imported successfully!")

... Libraries imported successfully!
```

Variables Terminal

5:16 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

UPLOAD THE DATASET

```
[17]
✓ 3m
from google.colab import files

print(" Please upload the Gurmukhi dataset ZIP file")

uploaded = files.upload()

ZIP_NAME = list(uploaded.keys())[0]

print("\n✅ Dataset uploaded successfully!")
print("File:", ZIP_NAME)

... Please upload the Gurmukhi dataset ZIP file
Choose Files Gurmukhi dataset.zip
Gurmukhi dataset.zip(application/x-zip-compressed) - 50064788 bytes, last modified: 9/24/2026 - 100% done
Saving Gurmukhi dataset.zip to Gurmukhi dataset (1).zip

✅ Dataset uploaded successfully!
File: Gurmukhi dataset (1).zip
```

Variables Terminal

5:24 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Dataset uploaded successfully!
File: Gurmukhi dataset (1).zip

EXTRACT THE DATASET

```
[18] ✓ 0s
EXTRACT_PATH = "/content/gurmukhi_dataset"

# Remove previous extraction if it exists
if os.path.exists(EXTRACT_PATH):
    shutil.rmtree(EXTRACT_PATH)

os.makedirs(EXTRACT_PATH)

# Extract ZIP
with zipfile.ZipFile(ZIP_NAME, "r") as zip_ref:
    zip_ref.extractall(EXTRACT_PATH)

print("Dataset extracted successfully!")
print("Extraction location:", EXTRACT_PATH)
```

Dataset extracted successfully!
Extraction location: /content/gurmukhi_dataset

Variables Terminal 5:26 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

FIND DATASET FOLDER

```
[11] ✓ 2s

# =====
# EXTRACT INNER DATASET ZIP
# =====

INNER_ZIP = os.path.join(
    EXTRACT_PATH,
    "Gurmukhi dataset",
    "archive.zip"
)

FINAL_DATASET_PATH = "/content/gurmukhi_final"

# Remove old folder if it exists
if os.path.exists(FINAL_DATASET_PATH):
    shutil.rmtree(FINAL_DATASET_PATH)

os.makedirs(FINAL_DATASET_PATH)

print("Extracting inner dataset ZIP...")
print("ZIP:", INNER_ZIP)

with zipfile.ZipFile(INNER_ZIP, "r") as zip_ref:
```

Variables Terminal 5:26 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5fBx8tuZH48gkrSLkRJ#scrollTo=IzI3PX6KJV8g

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

[11] 2s

with zipfile.ZipFile(INNER_ZIP, "r") as zip_ref:
zip_ref.extractall(FINAL_DATASET_PATH)

print("\n Inner dataset extracted successfully!")

Extracting inner dataset ZIP...
ZIP: /content/gurmukhi_dataset/Gurmukhi dataset/archive.zip

Inner dataset extracted successfully!

[12] 0s

=====
FIND THE 41 GURMUKHI CLASS FOLDERS
=====

print(" Finding character class folders...\n")

DATASET_PATH = None

for root, dirs, files_list in os.walk(FINAL_DATASET_PATH):

 class_folders_found = [
 d for d in dirs
 if d.isdigit() and 1 <= int(d) <= 41
]

 if len(class_folders_found) >= 35:

Variables Terminal

5:26 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5fBx8tuZH48gkrSLkRJ#scrollTo=IzI3PX6KJV8g

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

[12] 0s

if len(class_folders_found) >= 35:

 DATASET_PATH = root
 break

if DATASET_PATH is not None:

 class_folders = sorted(
 class_folders_found,
 key=lambda x: int(x)
)

 print(" DATASET FOUND!")
 print(" Path:", DATASET_PATH)
 print(" Number of classes:", len(class_folders))
 print(" Classes:", class_folders)

else:

 print(" Could not find the 41 class folders.")
 print("\nExtracted structure:")

 for root, dirs, files_list in os.walk(FINAL_DATASET_PATH):
 print(root)
 print("Folders:", dirs[:10])
 print("Files:", files_list[:10])

Variables Terminal

5:26 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
print(root)
print("Folders:", dirs[:10])
print("Files:", files_list[:10])
print("-" * 50)
```

Finding character class folders...

DATASET FOUND!
Path: /content/gurmukhi_final/b
Number of classes: 41
Classes: ['1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14', '15', '16', '17', '18', '19', '20', '21', '22', '23', '24', '25', '26', '27', '28', '29', '30', '31', '32', '33', '34', '35', '36', '37', '38', '39', '40', '41']

Gurmukhi Handwritten Character Recognition using Random Forest

COUNT IMAGES IN EACH CLASS

```
IMAGE_EXTENSIONS = (
    ".jpg",
    ".jpeg",
    ".png",
    ".bmp",
    ".webp"
)

class_counts = {}

for class_name in class_folders:
    class_path = os.path.join(
        DATASET_PATH,
        class_name
    )

    image_files = [
        file
        for file in os.listdir(class_path)
        if file.lower().endswith(IMAGE_EXTENSIONS)
    ]
```

Variables Terminal 5:28 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
[19] ✓ 0s ]

class_counts[class_name] = len(image_files)

# Create a DataFrame
class_distribution = pd.DataFrame(
    list(class_counts.items()),
    columns=["Class", "Number of Images"]
)

# Sort by class number
class_distribution["Class"] = class_distribution["Class"].astype(int)

class_distribution = class_distribution.sort_values(
    "Class"
).reset_index(drop=True)

class_distribution["Class"] = (
    class_distribution["Class"].astype(str)
)

# Display results
print("=" * 55)
print("          GURMUKHI CLASS DISTRIBUTION")
```

Variables Terminal 5:28 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
[19] ✓ 0s print("          GURMUKHI CLASS DISTRIBUTION")
print("=" * 55)

print(class_distribution.to_string(index=False))

print("\n" + "=" * 55)

total_images = class_distribution["Number of Images"].sum()

print(f"Total Images      : {total_images}")
print(f"Number of Classes : {len(class_distribution)}")
print(
    f"Average per Class : "
    f"{class_distribution['Number of Images'].mean():.2f}"
)

... =====
          GURMUKHI CLASS DISTRIBUTION
=====
Class Number of Images
1          297
2          299
3          294
4          295
5          297
6          296
7          293
```

Variables Terminal 5:28 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

5	297
6	296
7	293
8	297
9	295
10	294
11	292
12	296
13	296
14	299
15	294
16	298
17	297
18	296
19	294
20	297
21	296
22	295
23	299
24	296
25	298
26	297
27	291
28	297
29	292
30	299
31	296
32	298
33	294

Variables Terminal

5:28 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

32	298
33	294
34	294
35	297
36	299
37	296
38	297
39	295
40	293
41	293

=====

Total Images : 12128
Number of Classes : 41
Average per Class : 295.80

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

VISUALIZE CLASS DISTRIBUTION

```
[20] ✓ 0s
plt.figure(figsize=(16, 6))

sns.barplot(
    data=class_distribution,
    x="Class",
    y="Number of Images"
)

plt.title(
    "Distribution of Images Across Gurmukhi Character Classes",
    fontsize=18,
    fontweight="bold"
)

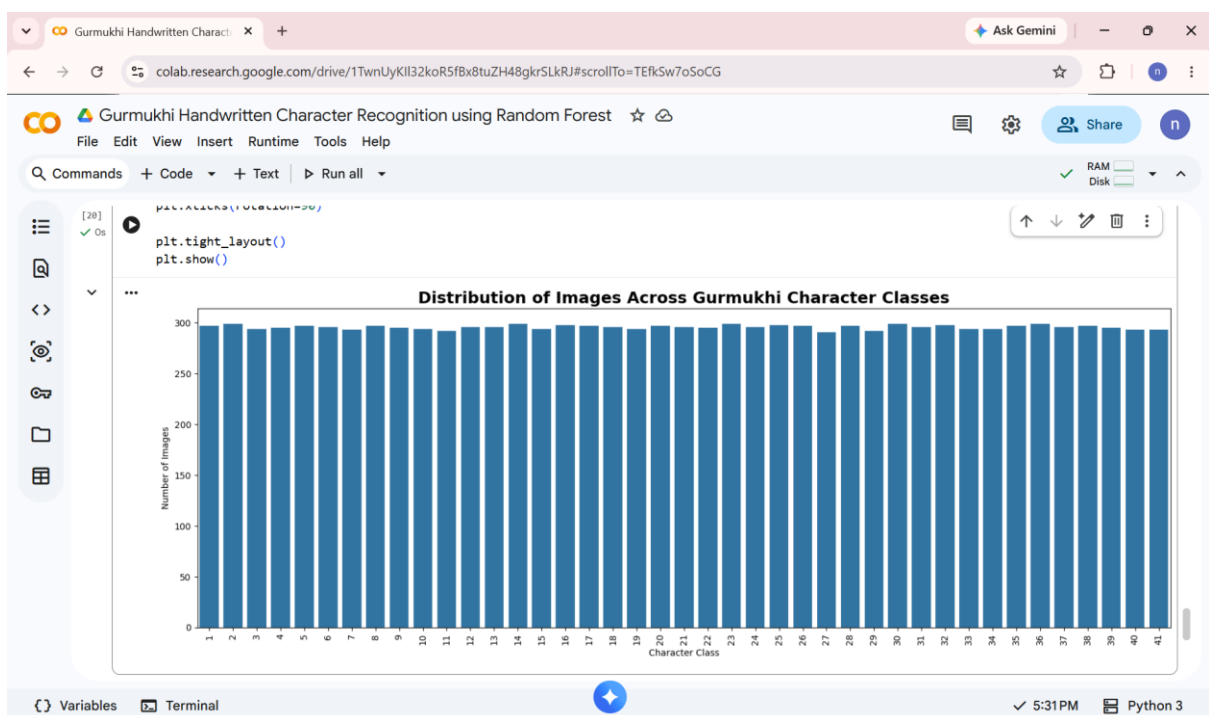
plt.xlabel("Character Class")
plt.ylabel("Number of Images")

plt.xticks(rotation=90)

plt.tight_layout()
plt.show()
```

Variables Terminal

5:31 PM Python 3



Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

DISPLAY SAMPLE IMAGES

```
[21] ✓ 3s
fig, axes = plt.subplots(
    4,
    8,
    figsize=(16, 9)
)

selected_classes = class_folders[:32]

for ax, class_name in zip(
    axes.ravel(),
    selected_classes
):

    class_path = os.path.join(
        DATASET_PATH,
        class_name
    )

    image_files = [
        file
        for file in os.listdir(class_path)
```

Variables Terminal

5:32 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

```
[21] ✓ 3s
    if file.lower().endswith(IMAGE_EXTENSIONS)
    ]

    image_file = random.choice(image_files)

    image_path = os.path.join(
        class_path,
        image_file
    )

    image = Image.open(image_path).convert("L")

    ax.imshow(
        image,
        cmap="gray"
    )

    ax.set_title(
        f"Class {class_name}"
    )

    ax.axis("off")

plt.suptitle(
    "Sample Handwritten Gurmukhi Characters",
    fontsize=18,
```

Variables Terminal

5:32 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
[21] ✓ 3s "Sample Handwritten Gurmukhi Characters",  
        fontsize=18,  
        fontweight="bold"  
        )  
  
        plt.tight_layout()  
        plt.show()
```

Sample Handwritten Gurmukhi Characters

Class 1	Class 2	Class 3	Class 4	Class 5	Class 6	Class 7	Class 8
Class 9	Class 10	Class 11	Class 12	Class 13	Class 14	Class 15	Class 16
Class 17	Class 18	Class 19	Class 20	Class 21	Class 22	Class 23	Class 24

Variables Terminal 5:32 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
...
```

Class 9	Class 10	Class 11	Class 12	Class 13	Class 14	Class 15	Class 16
Class 17	Class 18	Class 19	Class 20	Class 21	Class 22	Class 23	Class 24
Class 25	Class 26	Class 27	Class 28	Class 29	Class 30	Class 31	Class 32

Variables Terminal 5:32 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
[22] dimensions = []  
for class_name in class_folders:  
    class_path = os.path.join(  
        DATASET_PATH,  
        class_name  
    )  
    image_files = [  
        file  
        for file in os.listdir(class_path)  
        if file.lower().endswith(IMAGE_EXTENSIONS)  
    ]  
    for image_file in image_files[:10]:  
        image_path = os.path.join(  
            class_path,  
            image_file  
        )
```

Variables Terminal 5:35 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
[22] class_path,  
image_file  
)  
  
try:  
    image = Image.open(image_path)  
    dimensions.append(image.size)  
  
except:  
    pass  
  
dimension_counts = pd.Series(  
    dimensions  
)  
.value_counts()  
  
print(dimension_counts.head(15))  
  
... (256, 256) 410  
Name: count, dtype: int64
```

Variables Terminal 5:35 PM Python 3

Gurmukhi Handwritten Character x +Ask Gemini

colab.research.google.com/drive/1TwnUyKil32koR5fBx8tuZH48gkrSLkRJ#scrollTo=NLrzTdERUYcp

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAMDisk

IMAGE PREPROCESSING

[23] 0s

Image preprocessing configuration

IMAGE_SIZE = (32, 32)

print("Target image size:", IMAGE_SIZE)

print("Number of pixel features:", IMAGE_SIZE[0] * IMAGE_SIZE[1])

Target image size: (32, 32)

Number of pixel features: 1024

LOAD AND PREPROCESS IMAGES

[24] 16s

X_images = []

y_labels = []

image_paths = []

for class_name in class_folders:

class_path = os.path.join(

VariablesTerminal

5:39 PMPython 3

Gurmukhi Handwritten Character x +Ask Gemini

colab.research.google.com/drive/1TwnUyKil32koR5fBx8tuZH48gkrSLkRJ#scrollTo=NLrzTdERUYcp

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAMDisk

[24] 16s

class_path = os.path.join(

DATASET_PATH,

class_name

)

image_files = [

file

for file in os.listdir(class_path)

if file.lower().endswith(IMAGE_EXTENSIONS)

]

for image_file in image_files:

image_path = os.path.join(

class_path,

image_file

)

try:

image = Image.open(

image_path

).convert("L")

image = image.resize(

IMAGE_SIZE,

Image.Resampling.LANCZOS

,

VariablesTerminal

5:39 PMPython 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

[24] ✓ 16s

```
        Image.Resampling.LANCZOS

        image_array = np.array(
            image,
            dtype=np.float32
        )

        image_array = image_array / 255.0

        X_images.append(image_array)
        y_labels.append(class_name)
        image_paths.append(image_path)

    except Exception as e:
        print(
            f"Error loading {image_path}: {e}"
        )

X_images = np.array(X_images)
y_labels = np.array(y_labels)

print("Images loaded:", len(X_images))
print("Image shape:", X_images.shape)
print("Labels shape:", y_labels.shape)
```

Variables

Terminal

5:39 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

L=2 ✓ 16s

```
print("Labels shape:", y_labels.shape)
```

Images loaded: 12128
Image shape: (12128, 32, 32)
Labels shape: (12128,)

Visualize Preprocessed Images

[25] ✓ 1s

```
fig, axes = plt.subplots(
    3,
    6,
    figsize=(12, 7)
)

sample_indices = np.random.choice(
    len(X_images),
    18,
    replace=False
)

for ax, index in zip(
    axes.ravel(),
    sample_indices
):
```

Variables

Terminal

5:40 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKI32koR5fBx8tuZH48gkrSLkRJ#scrollTo=je6HbxEXU06c

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

[25] 1s

```
ax.imshow(
    X_images[index],
    cmap="gray"
)

ax.set_title(
    f"Class {y_labels[index]}"
)

ax.axis("off")

plt.suptitle(
    "Preprocessed Gurmukhi Character Images",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()
plt.show()
```

...

Preprocessed Gurmukhi Character Images

Class 21Class 34Class 8Class 5Class 40Class 38

Variables Terminal

5:40 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKI32koR5fBx8tuZH48gkrSLkRJ#scrollTo=je6HbxEXU06c

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

1s

...

Preprocessed Gurmukhi Character Images

Class 21Class 34Class 8Class 5Class 40Class 38

Variables Terminal

5:40 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

Class 41 Class 25 Class 5 Class 37 Class 5 Class 18

Class 37 Class 22 Class 6 Class 1 Class 39 Class 27

Variables Terminal

5:43 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

SPLIT THE DATASET

```
from sklearn.model_selection import train_test_split

X_train_img, X_test_img, y_train, y_test, paths_train, paths_test = train_test_split(
    X_images,
    y_labels,
    image_paths,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_labels
)

print("Training images:", len(X_train_img))
print("Testing images:", len(X_test_img))
print("Training labels:", len(y_train))
print("Testing labels:", len(y_test))
```

Training images: 9702
Testing images: 2426
Training labels: 9702
Testing labels: 2426

Variables Terminal

5:43 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

GENERATE PREDICTIONS

```
[29] y_pred_raw = rf_raw.predict(X_test_raw)

print("Predictions generated successfully.")
print("Number of predictions:", len(y_pred_raw))
```

... Predictions generated successfully.
Number of predictions: 2426

BASELINE MODEL EVALUATION

```
[30] from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

Variables Terminal 5:53 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

```
[30] accuracy_raw = accuracy_score(
    y_test,
    y_pred_raw
)

precision_raw = precision_score(
    y_test,
    y_pred_raw,
    average="macro"
)

recall_raw = recall_score(
    y_test,
    y_pred_raw,
    average="macro"
)

f1_raw = f1_score(
    y_test,
    y_pred_raw,
    average="macro"
)

print("=" * 45)
print("RANDOM FOREST BASELINE RESULTS")
print("=" * 45)
```

Variables Terminal 5:53 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKil32koR5fBx8tuZH48gkrSLkRJ#scrollTo=BFRIhAGHXo0v

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[30] 0s

```
print("=" * 45)
print("RANDOM FOREST BASELINE RESULTS")
print("=" * 45)

print(f"Accuracy : {accuracy_raw:.4f}")
print(f"Precision : {precision_raw:.4f}")
print(f"Recall : {recall_raw:.4f}")
print(f"F1-Score : {f1_raw:.4f}")

...
=====
RANDOM FOREST BASELINE RESULTS
=====
Accuracy : 0.9575
Precision : 0.9580
Recall : 0.9576
F1-Score : 0.9574
```

Variables Terminal 5:53 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKil32koR5fBx8tuZH48gkrSLkRJ#scrollTo=zEuTT_FjYaqd

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

HOG FEATURE EXTRACTION

[31] 27s

```
from skimage.feature import hog

def extract_hog_features(images):
    features = []

    for image in images:

        hog_features = hog(
            image,
            orientations=9,
            pixels_per_cell=(4, 4),
            cells_per_block=(2, 2),
            block_norm="L2-Hys"
        )

        features.append(hog_features)

    return np.array(features)

X_train_hog = extract_hog_features(X_train_img)
X_test_hog = extract_hog_features(X_test_img)
```

Variables Terminal 5:57 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[31] ✓ 27s

```
print("Training HOG feature shape:", X_train_hog.shape)
print("Testing HOG feature shape:", X_test_hog.shape)
```

Training HOG feature shape: (9702, 1764)
Testing HOG feature shape: (2426, 1764)

RANDOM FOREST WITH HOG FEATURES

[32] ✓ 2m

```
rf_hog = RandomForestClassifier(
    n_estimators=200,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

rf_hog.fit(
    X_train_hog,
    y_train
)

print("Random Forest with HOG features trained successfully.")
```

... Random Forest with HOG features trained successfully.

Variables Terminal

✓ 6:03 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

GENERATE HOG PREDICTIONS

[33] ✓ 0s

```
y_pred_hog = rf_hog.predict(
    X_test_hog
)

print("HOG predictions generated successfully.")
print("Number of predictions:", len(y_pred_hog))
```

HOG predictions generated successfully.
Number of predictions: 2426

HOG MODEL EVALUATION

[34] ✓ 0s

```
accuracy_hog = accuracy_score(
    y_test,
    y_pred_hog
)

precision_hog = precision_score(
    y_test,
```

Variables Terminal

✓ 6:05 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKil32koR5f8x8tuZH48gkrSLkRJ#scrollTo=k9lbTDMsb8MO

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[35] 0s

```
accuracy_raw,
accuracy_hog

],
"Precision": [
precision_raw,
precision_hog
],
"Recall": [
recall_raw,
recall_hog
],
"F1-Score": [
f1_raw,
f1_hog
]
}]

display(
model_comparison.style.format({
"Accuracy": "{:.2%}",
"Precision": "{:.2%}",
"Recall": "{:.2%}",
"F1-Score": "{:.2%}"
})
)
```

Variables Terminal 6:08 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKil32koR5f8x8tuZH48gkrSLkRJ#scrollTo=KVPb6A8KbSUV

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[35] 0s

```
"""
"F1-Score": [
f1_raw,
f1_hog
]
}]

display(
model_comparison.style.format({
"Accuracy": "{:.2%}",
"Precision": "{:.2%}",
"Recall": "{:.2%}",
"F1-Score": "{:.2%}"
})
)
```

	Model	Accuracy	Precision	Recall	F1-Score
0	Random Forest + Raw Pixels	95.75%	95.80%	95.76%	95.74%
1	Random Forest + HOG	78.52%	79.03%	78.52%	78.46%

MODEL PERFORMANCE VISUALIZATION

Variables Terminal 6:08 PM Python 3

Gurmukhi Handwritten Character x +Ask Gemini

colab.research.google.com/drive/1TwnUyKII32koR5fBx8tuZH48gkrSLkRJ#scrollTo=KVPb6A8KbSUV

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAMDisk

MODEL PERFORMANCE VISUALIZATION

```
[36]
✓ 0s
metrics = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1-Score"
]

x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(11, 6))

bars1 = ax.bar(
    x - width / 2,
    model_comparison.iloc[0][metrics],
    width,
    label="Raw Pixels + Random Forest"
)

bars2 = ax.bar(
    x + width / 2,
    model_comparison.iloc[1][metrics],
    width,
    label="HOG + Random Forest"
)

ax.set_title(
    "Comparison of Random Forest Feature Representations",
    fontsize=18,
    fontweight="bold"
)

ax.set_ylabel("Score")
ax.set_xticks(x)
ax.set_xticklabels(metrics)

ax.set_ylim(0, 1.05)

ax.legend()

ax.bar_label(
    bars1,
    fmt="%.3f",
    padding=3
)

ax.bar_label(
    bars2,
```

Variables Terminal

6:09 PM Python 3

Gurmukhi Handwritten Character x +Ask Gemini

colab.research.google.com/drive/1TwnUyKII32koR5fBx8tuZH48gkrSLkRJ#scrollTo=KVPb6A8KbSUV

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAMDisk

```
[36]
✓ 0s
width,
label="HOG + Random Forest"
)

ax.set_title(
    "Comparison of Random Forest Feature Representations",
    fontsize=18,
    fontweight="bold"
)

ax.set_ylabel("Score")
ax.set_xticks(x)
ax.set_xticklabels(metrics)

ax.set_ylim(0, 1.05)

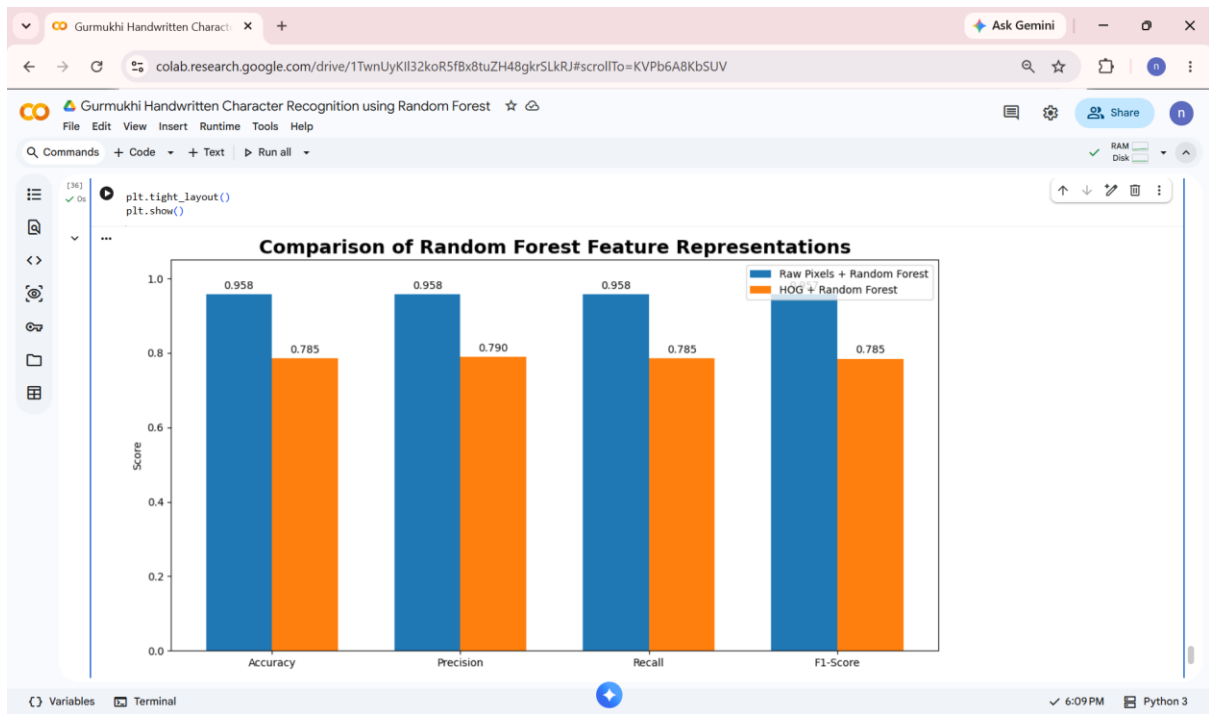
ax.legend()

ax.bar_label(
    bars1,
    fmt="%.3f",
    padding=3
)

ax.bar_label(
    bars2,
```

Variables Terminal

6:09 PM Python 3



Gurmukhi Handwritten Character Recognition using Random Forest

MODEL OPTIMIZATION

```
from sklearn.model_selection import RandomizedSearchCV

param_grid = {
    "n_estimators": [100, 200, 300, 400],
    "max_depth": [None, 20, 30, 40],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2"]
}

rf_tuning = RandomForestClassifier(
    random_state=RANDOM_STATE,
    n_jobs=-1
)

random_search = RandomizedSearchCV(
    estimator=rf_tuning,
    param_distributions=param_grid,
    n_iter=10,
    cv=3,
    scoring="accuracy",
```

Variables Terminal 6:24 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
verbose=1

random_search.fit(
    X_train_raw,
    y_train
)

print("Hyperparameter tuning completed.")
print("\nBest parameters:")
print(random_search.best_params_)

print(
    f"\nBest cross-validation accuracy: "
    f"{random_search.best_score_:.4f}"
)
```

... Fitting 3 folds for each of 10 candidates, totalling 30 fits
/usr/local/lib/python3.13/dist-packages/joblib/externals/loky/process_executor.py:787: UserWarning: A worker stopped while some jobs were given to it. This could be caused by a canceled job or by a worker that has been killed.
warnings.warn(
Hyperparameter tuning completed.

Best parameters:
{'n_estimators': 400, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'log2', 'max_depth': 30}

Best cross-validation accuracy: 0.9477

Gurmukhi Handwritten Character Recognition using Random Forest

```
rf_tuned = random_search.best_estimator_

y_pred_tuned = rf_tuned.predict(
    X_test_raw
)

accuracy_tuned = accuracy_score(
    y_test,
    y_pred_tuned
)

precision_tuned = precision_score(
    y_test,
    y_pred_tuned,
    average="macro"
)

recall_tuned = recall_score(
    y_test,
    y_pred_tuned,
    average="macro"
)
```

Gurmukhi Handwritten Character Recognition using Random Forest

```

[38] ✓ 1s
)
    average="macro"

f1_tuned = f1_score(
    y_test,
    y_pred_tuned,
    average="macro"
)

print("=" * 45)
print("TUNED RANDOM FOREST RESULTS")
print("=" * 45)

print(f"Accuracy : {accuracy_tuned:.4f}")
print(f"Precision : {precision_tuned:.4f}")
print(f"Recall : {recall_tuned:.4f}")
print(f"F1-Score : {f1_tuned:.4f}")

...
=====
TUNED RANDOM FOREST RESULTS
=====
Accuracy : 0.9654
Precision : 0.9665
Recall : 0.9654
F1-Score : 0.9653

```

Variables Terminal 6:26 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

CLASSIFICATION REPORT

```

[39] ✓ 0s
from sklearn.metrics import classification_report

report = classification_report(
    y_test,
    y_pred_tuned,
    digits=4
)

print(report)

```

	precision	recall	f1-score	support
1	1.0000	1.0000	1.0000	59
10	0.9219	1.0000	0.9593	59
11	0.9508	1.0000	0.9748	58
12	0.9828	0.9661	0.9744	59
13	1.0000	0.9322	0.9649	59
14	1.0000	0.9333	0.9655	60
15	1.0000	0.8305	0.9074	59
16	0.9516	0.9833	0.9672	60
17	0.9672	1.0000	0.9833	59
18	1.0000	0.9831	0.9915	59
19	0.9222	1.0000	0.9677	60

Variables Terminal 6:28 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5fBx8tuZH48gkrSLkRJ#scrollTo=c28QWDcIf0NI

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

19 0.9365 1.0000 0.9672 59

2 0.9825 0.9333 0.9573 60

20 0.9492 0.9492 0.9492 59

21 0.9355 0.9831 0.9587 59

22 0.9818 0.9153 0.9474 59

23 0.8906 0.9500 0.9194 60

24 0.9508 0.9831 0.9667 59

25 0.9828 0.9500 0.9661 60

26 0.9667 0.9831 0.9748 59

27 0.9649 0.9483 0.9565 58

28 0.9508 0.9667 0.9587 60

29 1.0000 1.0000 1.0000 58

3 0.9833 1.0000 0.9916 59

30 1.0000 0.9833 0.9916 60

31 0.9833 1.0000 0.9916 59

32 0.9492 0.9333 0.9412 60

33 0.8983 0.8983 0.8983 59

34 1.0000 0.9831 0.9915 59

35 0.9833 1.0000 0.9916 59

36 1.0000 0.9833 0.9916 60

37 1.0000 0.9661 0.9828 59

38 0.9636 0.8983 0.9298 59

39 0.8939 1.0000 0.9440 59

4 1.0000 0.9661 0.9828 59

40 0.9643 0.9153 0.9391 59

41 0.9355 0.9831 0.9587 59

5 0.9194 0.9500 0.9344 60

6 0.9833 1.0000 0.9916 59

7 0.9818 0.9551 0.9744 59

Variables Terminal

6:28 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5fBx8tuZH48gkrSLkRJ#scrollTo=wEq2dMB7gMCo

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

5 0.9194 0.9500 0.9344 60

6 0.9833 1.0000 0.9916 59

7 0.9828 0.9661 0.9744 59

8 0.9206 0.9831 0.9508 59

9 1.0000 0.9831 0.9915 59

accuracy 0.9665 0.9654 0.9653 2426

macro avg 0.9665 0.9654 0.9653 2426

weighted avg 0.9665 0.9654 0.9653 2426

CONFUSION MATRIX

[40] ✓ 1s

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(
y_test,
y_pred_tuned,
labels=class_folders
)

plt.figure(figsize=(14, 12))

sns.heatmap(

Variables Terminal

6:30 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKII32koR5fBx8tuZH48gkrSLkRJ#scrollTo=wEq2dMB7gMCo

colab.research.google.com/drive/1TwnUyKII32koR5fBx8tuZH48gkrSLkRJ#scrollTo=wEq2dMB7gMCo

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

[40] 1s

```
sns.heatmap(
    cm,
    annot=False,
    cmap="Blues",
    xticklabels=class_folders,
    yticklabels=class_folders
)

plt.title(
    "Confusion Matrix - Tuned Random Forest",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel(
    "Predicted Class",
    fontsize=13
)

plt.ylabel(
    "Actual Class",
    fontsize=13
)

plt.tight_layout()
plt.show()
```

Variables Terminal

6:30 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKII32koR5fBx8tuZH48gkrSLkRJ#scrollTo=wEq2dMB7gMCo

colab.research.google.com/drive/1TwnUyKII32koR5fBx8tuZH48gkrSLkRJ#scrollTo=wEq2dMB7gMCo

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

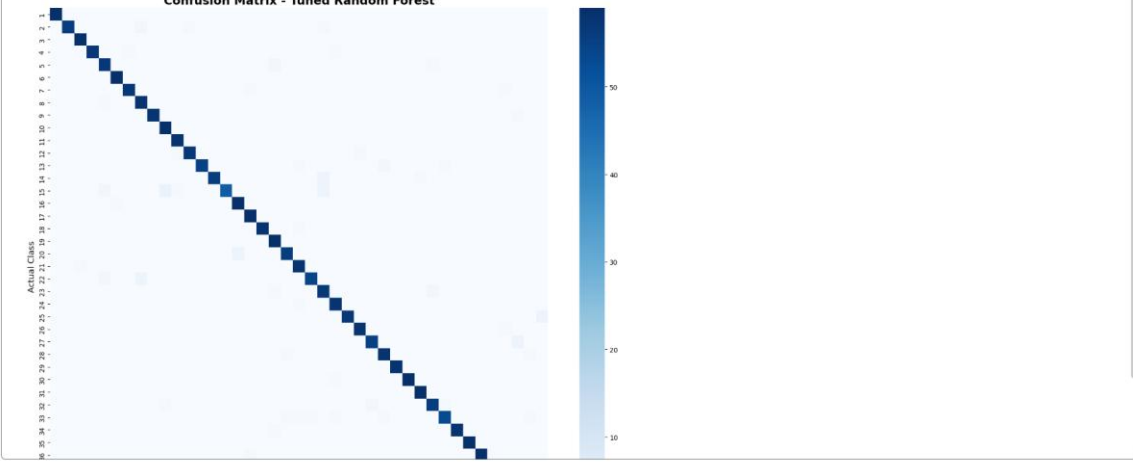
RAM Disk

[40] 1s

```
fontsize=13
plt.tight_layout()
plt.show()
```

Variables Terminal

6:30 PM Python 3



The figure is a heatmap titled "Confusion Matrix - Tuned Random Forest". The x-axis is labeled "Predicted Class" and the y-axis is labeled "Actual Class". Both axes have 36 tick labels representing Gurmukhi characters. The diagonal elements, where predicted equals actual, are dark blue, indicating high accuracy. The off-diagonal elements are lighter blue, indicating some misclassifications. A color bar on the right indicates the count of instances, ranging from 0 to 50.

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKlI32koR5fBx8tuZH48gkrSLkRJ#scrollTo=mGvoW5ZbhX9w

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

[43] 1s

```
f"Predicted: {y_pred_tuned[index]}",
    fontsize=10
)

ax.axis("off")

plt.suptitle(
    "Examples of Misclassified Gurmukhi Characters",
    fontsize=10,
    fontweight="bold"
)

plt.tight_layout()
plt.show()
```

RAM

Disk

Examples of Misclassified Gurmukhi Characters

Actual: 14
Predicted: 31

Actual: 36
Predicted: 17

Actual: 33
Predicted: 22

Actual: 15
Predicted: 10

Actual: 13
Predicted: 28

Variables

Terminal

6:35 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKlI32koR5fBx8tuZH48gkrSLkRJ#scrollTo=mGvoW5ZbhX9w

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM

Disk

Examples of Misclassified Gurmukhi Characters

Actual: 14
Predicted: 31

Actual: 36
Predicted: 17

Actual: 33
Predicted: 22

Actual: 15
Predicted: 10

Actual: 13
Predicted: 28

Actual: 38
Predicted: 2

Actual: 32
Predicted: 33

Actual: 12
Predicted: 26

Actual: 15
Predicted: 23

Actual: 33
Predicted: 21

Actual: 30
Predicted: 24

Actual: 15
Predicted: 23

Actual: 40
Predicted: 11

Actual: 25
Predicted: 41

Actual: 33
Predicted: 28

Variables

Terminal

6:35 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5f8x8tuZH48gkrSLkRJ#scrollTo=mGvoW5ZbhX9w

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

Actual: 30
Predicted: 24

Actual: 15
Predicted: 23

Actual: 40
Predicted: 11

Actual: 25
Predicted: 41

Actual: 33
Predicted: 28

Actual: 37
Predicted: 33

Actual: 33
Predicted: 24

Actual: 38
Predicted: 26

Actual: 16
Predicted: 6

Actual: 15
Predicted: 10

Variables Terminal

6:35 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5f8x8tuZH48gkrSLkRJ#scrollTo=_qeH05tDg974

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

MOST FREQUENTLY CONFUSED CLASSES

```
[41] 0s
confusion_pairs = []

for i in range(len(class_folders)):
    for j in range(len(class_folders)):
        if i != j and cm[i, j] > 0:
            confusion_pairs.append({
                "Actual Class": class_folders[i],
                "Predicted Class": class_folders[j],
                "Misclassifications": cm[i, j]
            })

confusion_pairs_df = pd.DataFrame(
    confusion_pairs
)

confusion_pairs_df = confusion_pairs_df.sort_values(
    "Misclassifications",
    ascending=False
).reset_index(drop=True)
```

Variables Terminal

6:33 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5f8x8tuZH48gkrSLkRJ#scrollTo=_qeH05tDg974

☆

🔍

👤

🔗

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

🔍 Commands + Code + Text ▶ Run all

✓ RAM Disk

📄

🔍

🔗

📁

📊

[41] ✓ 0s

```
confusion_pairs_df = confusion_pairs_df.sort_values(
    "Misclassifications",
    ascending=False
).reset_index(drop=True)

print(
    confusion_pairs_df.head(10).to_string(index=False)
)
```

...	Actual Class	Predicted Class	Misclassifications
	15	10	4
	14	23	3
	15	23	3
	20	16	3
	27	39	3
	25	41	3
	22	8	3
	38	39	3
	15	5	2
	13	28	2

📄 Variables

📄 Terminal

🔗

✓ 6:33 PM

📄 Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5f8x8tuZH48gkrSLkRJ#scrollTo=Exlu2RjllGQm

☆

🔍

👤

🔗

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

🔍 Commands + Code + Text ▶ Run all

✓ RAM Disk

📄

🔍

🔗

📁

📊

TOP CONFUSION PAIRS

[45] ✓ 0s

```
top_confusions = confusion_pairs_df.head(10).copy()

top_confusions["Pair"] = (
    top_confusions["Actual Class"].astype(str)
    + " → "
    + top_confusions["Predicted Class"].astype(str)
)

plt.figure(figsize=(11, 6))

plt.barh(
    top_confusions["Pair"][::-1],
    top_confusions["Misclassifications"][::-1]
)

plt.title(
    "Top Character Confusions",
    fontsize=18,
    fontweight="bold"
)
```

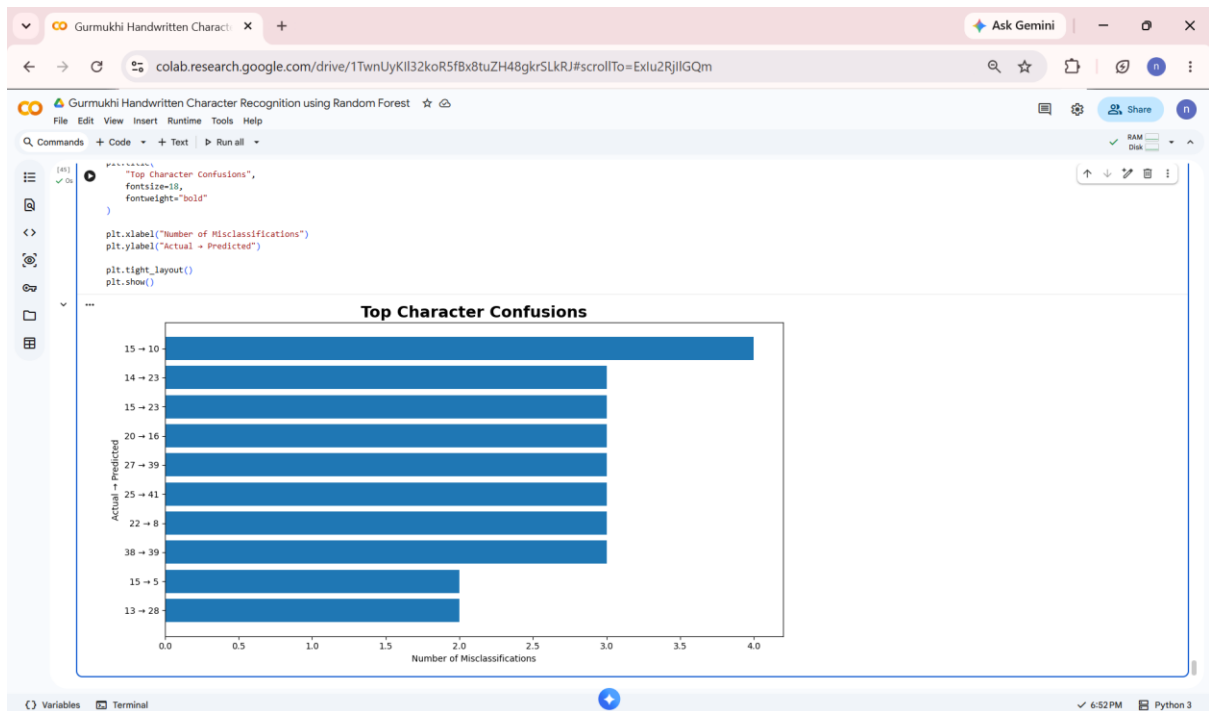
📄 Variables

📄 Terminal

🔗

✓ 6:52 PM

📄 Python 3



Gurmukhi Handwritten Character Recognition using Random Forest

```
[46] ✓ 18  
prediction_probabilities = rf_tuned.predict_proba(X_test_raw)  
  
prediction_confidence = np.max(  
    prediction_probabilities,  
    axis=1  
)  
  
confidence_results = pd.DataFrame({  
    "Actual Class": y_test,  
    "Predicted Class": y_pred_tuned,  
    "Confidence": prediction_confidence  
})  
  
confidence_results["Correct"] = (  
    confidence_results["Actual Class"]  
    == confidence_results["Predicted Class"]  
)  
  
print(  
    f"Average prediction confidence: "  
    f"{prediction_confidence.mean():.2%}"  
)
```

Gurmukhi Handwritten Character Recognition using Random Forest

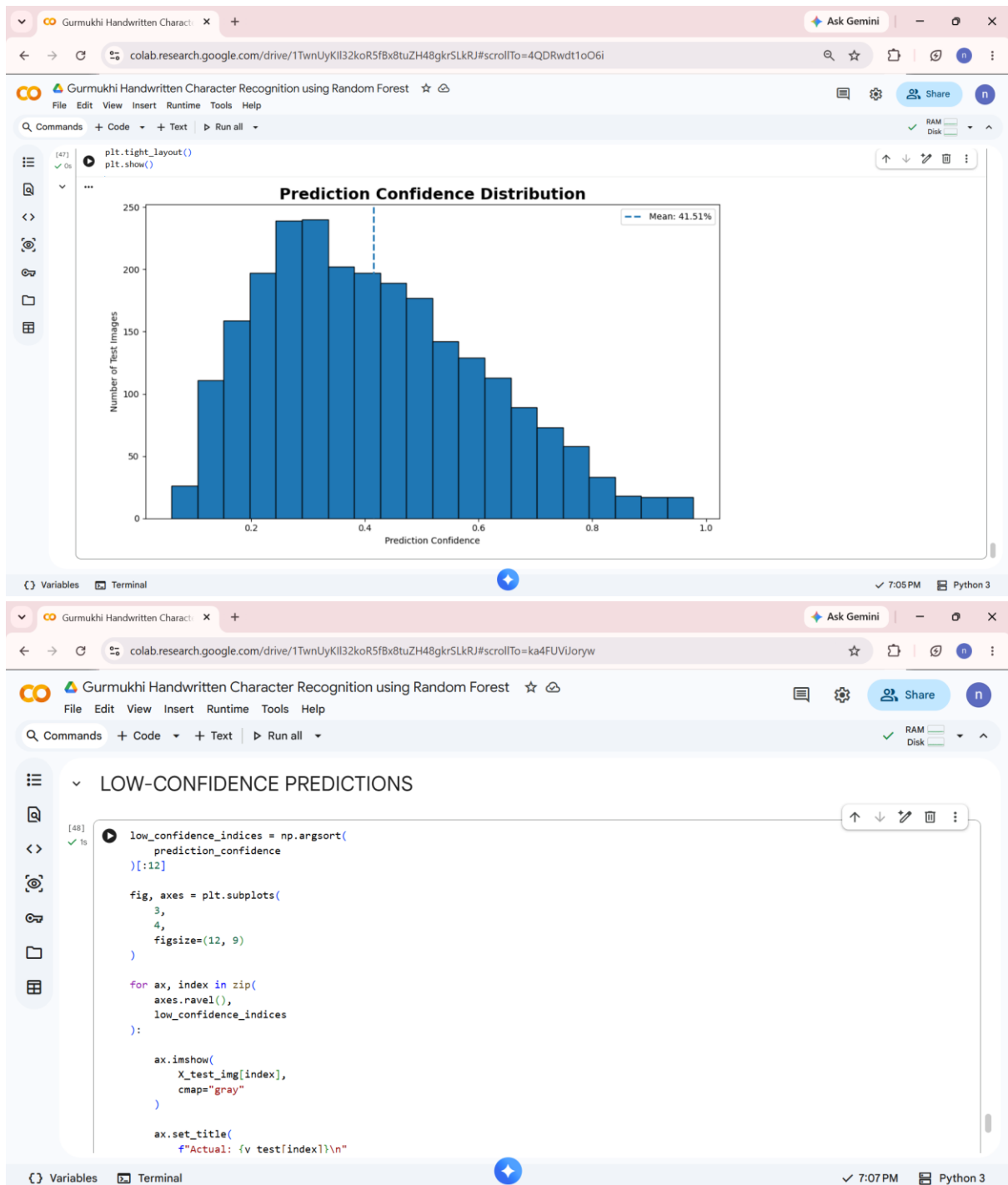
```
[46] ✓ 1s  
    }  
    "Confidence": prediction_confidence  
    }  
    }  
    confidence_results["Correct"] = (  
        confidence_results["Actual Class"]  
        == confidence_results["Predicted Class"]  
    )  
    print(  
        f"Average prediction confidence: "  
        f"{prediction_confidence.mean():.2%}"  
    )  
    print(  
        f"Correct predictions: "  
        f"{confidence_results['Correct'].mean():.2%}"  
    )  
    ...  
    Average prediction confidence: 41.51%  
    Correct predictions: 96.54%
```

Variables Terminal 7:04 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
[47] ✓ 0s  
plt.figure(figsize=(10, 6))  
  
plt.hist(  
    prediction_confidence,  
    bins=20,  
    edgecolor="black"  
)  
  
plt.axvline(  
    prediction_confidence.mean(),  
    linestyle="--",  
    linewidth=2,  
    label=f"Mean: {prediction_confidence.mean():.2%}"  
)  
  
plt.title(  
    "Prediction Confidence Distribution",  
    fontsize=18,  
    fontweight="bold"  
)  
  
plt.xlabel("Prediction Confidence")  
plt.ylabel("Number of Test Images")  
  
plt.legend()  
  
plt.tight_layout()  
plt.show()
```

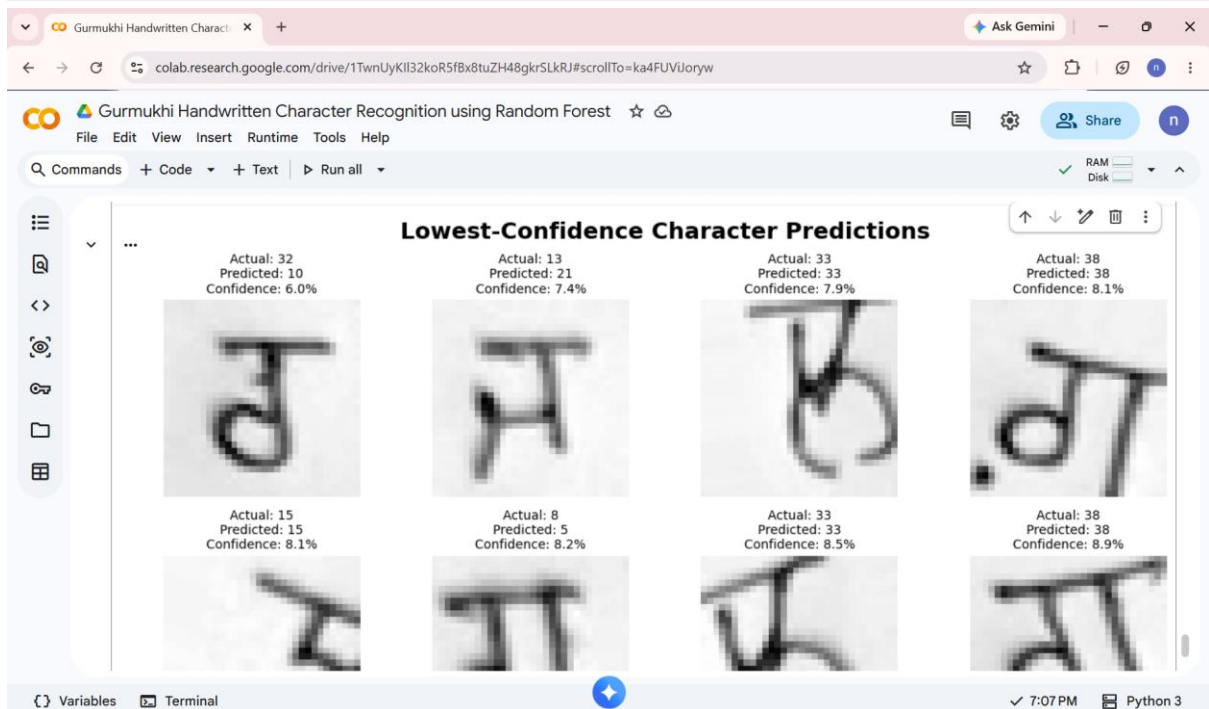
Variables Terminal 7:05 PM Python 3



Gurmukhi Handwritten Character Recognition using Random Forest

```
low_confidence_indices = ...  
ax.imshow(X_test_img[index], cmap="gray")  
ax.set_title(f"Actual: {y_test[index]}\nPredicted: {y_pred_tuned[index]}\nConfidence: {prediction_confidence[index]:.1%}", fontsize=10)  
ax.axis("off")  
plt.suptitle("Lowest-Confidence Character Predictions", fontsize=10, fontweight="bold")  
plt.tight_layout()  
plt.show()
```

Variables Terminal 7:07 PM Python 3



Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

RAM Disk

Actual	Predicted	Confidence
15	15	8.1%
8	5	8.2%
33	33	8.5%
33	33	8.9%
21	3	9.1%
38	38	9.1%
27	27	9.3%
33	28	9.3%

Variables Terminal 7:07 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

RAM Disk

UNSEEN CHARACTER PREDICTION

```
[49] ✓ 40s
from google.colab import files
from PIL import Image
import io

uploaded = files.upload()

file_name = list(uploaded.keys())[0]

image = Image.open(
    io.BytesIO(uploaded[file_name])
).convert("L")

print("Uploaded image:", file_name)
print("Original image size:", image.size)
```

Choose Files aug_0_976.jpg
aug_0_976.jpg(image/jpeg) - 3829 bytes, last modified: 9/24/2026 - 100% done
Saving aug_0_976.jpg to aug_0_976.jpg
Uploaded image: aug_0_976.jpg
Original image size: (256, 256)

Variables Terminal 7:12 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

PREPROCESS UNSEEN IMAGE

```
[50] ✓ 0s
image_resized = image.resize((32, 32))

image_array = np.array(
    image_resized,
    dtype=np.float32
) / 255.0

image_input = image_array.reshape(
    1,
    -1
)

print("Processed image shape:", image_input.shape)
```

... Processed image shape: (1, 1024)

Variables Terminal 7:13 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

PREDICT UNSEEN CHARACTER

```
[51] ✓ 0s
unseen_prediction = rf_tuned.predict(
    image_input
)[0]

unseen_probabilities = rf_tuned.predict_proba(
    image_input
)[0]

unseen_confidence = np.max(
    unseen_probabilities
)

print("=" * 45)
print("UNSEEN CHARACTER PREDICTION")
print("=" * 45)

print("Predicted Class :", unseen_prediction)
print(
    f"Confidence      : {unseen_confidence:.2%}"
)
```

Variables Terminal 7:14 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5fbx8tuZH48gkrSLkRJ#scrollTo=k3xOswkNqhxM

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

[51] 0s

```
f"Confidence : {unseen_confidence:.2%}"
```

...

=====

UNSEEN CHARACTER PREDICTION

=====

Predicted Class : 30

Confidence : 86.45%

+ Code + Text

VISUALIZE UNSEEN PREDICTION

[52] 0s

```
plt.figure(figsize=(5, 5))

plt.imshow(
    image_array,
    cmap="gray"
)

plt.title(
    f"Predicted Class: {unseen_prediction}\n"
    f"Confidence: {unseen_confidence:.2%}",
    fontsize=18,
    fontweight="bold"
)
```

Variables Terminal

7:15 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5fbx8tuZH48gkrSLkRJ#scrollTo=k3xOswkNqhxM

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

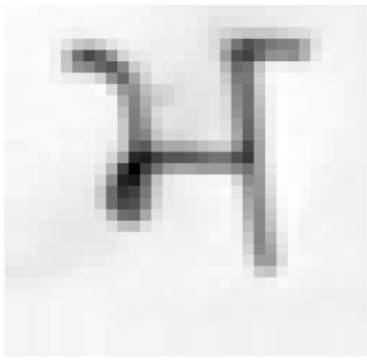
RAM Disk

[52] 0s

```
plt.axis("off")
plt.show()
```

...

Predicted Class: 30
Confidence: 86.45%



Variables Terminal

7:15 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

TOP 5 PREDICTIONS

```
[53] ✓ 0s
top_5_indices = np.argsort(
    unseen_probabilities
)[-5:][::-1]

top_5_classes = [
    rf_tuned.classes_[i]
    for i in top_5_indices
]

top_5_probabilities = [
    unseen_probabilities[i]
    for i in top_5_indices
]

top_5_results = pd.DataFrame({
    "Class": top_5_classes,
    "Probability": top_5_probabilities
})

top_5_results["Probability"] = (
    top_5_results["Probability"]
```

Variables Terminal 7:17 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
[53] ✓ 0s
top_5_results = pd.DataFrame({
    "Class": top_5_classes,
    "Probability": top_5_probabilities
})

top_5_results["Probability"] = (
    top_5_results["Probability"]
    .map(lambda x: f"{x:.2%}")
)

display(top_5_results)
```

	Class	Probability
0	30	86.45%
1	39	3.29%
2	9	2.47%
3	13	1.56%
4	28	1.23%

Next steps: [Generate code with top_5_results](#) [New interactive sheet](#)

Variables Terminal 7:17 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5f8x8tuZH48gkrSLkRJ#scrollTo=fyLp4HvIrNxH

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

TOP 5 PREDICTION PROBABILITIES

```
[54]
plt.figure(figsize=(9, 5))

plt.barh(
    top_5_results["Class"].astype(str)[::-1],
    top_5_probabilities[::-1]
)

plt.xlabel("Prediction Probability")
plt.ylabel("Character Class")

plt.title(
    "Top 5 Predicted Gurmukhi Classes",
    fontsize=18,
    fontweight="bold"
)

plt.xlim(0, 1)

plt.tight_layout()
plt.show()
```

Variables Terminal

7:18 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5f8x8tuZH48gkrSLkRJ#scrollTo=fyLp4HvIrNxH

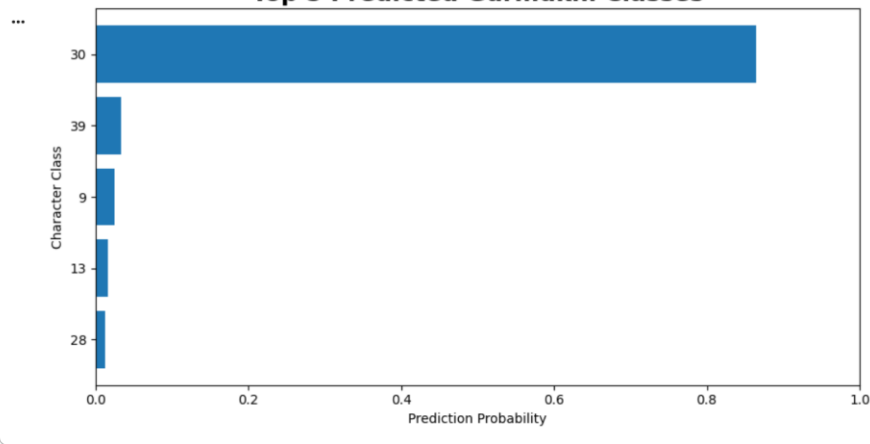
Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

Top 5 Predicted Gurmukhi Classes



Character Class	Prediction Probability
30	0.85
39	0.05
9	0.03
13	0.02
28	0.01

Variables Terminal

7:18 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5f8x8tuZH48gkrSLkRJ#scrollTo=8ZgZSFuZr_GD

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

CLASS-WISE PERFORMANCE

[55] 0s

```
from sklearn.metrics import precision_recall_fscore_support

precision_class, recall_class, f1_class, support_class = (
    precision_recall_fscore_support(
        y_test,
        y_pred_tuned,
        labels=class_folders,
        zero_division=0
    )
)

class_performance = pd.DataFrame({
    "Class": class_folders,
    "Precision": precision_class,
    "Recall": recall_class,
    "F1-Score": f1_class,
    "Support": support_class
})

display(
    class_performance.style.format({
        "Precision": "{:.2%}",
```

Variables Terminal

7:22 PM Python 3

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKIi32koR5f8x8tuZH48gkrSLkRJ#scrollTo=8ZgZSFuZr_GD

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

["Precision": "{:.2%}",
"Recall": "{:.2%}",
"F1-Score": "{:.2%}"]

...

10	11	95.08%	100.00%	97.48%	58
11	12	98.28%	96.61%	97.44%	59
12	13	100.00%	93.22%	96.49%	59
13	14	100.00%	93.33%	96.55%	60
14	15	100.00%	83.05%	90.74%	59
15	16	95.16%	98.33%	96.72%	60
16	17	96.72%	100.00%	98.33%	59
17	18	100.00%	98.31%	99.15%	59
18	19	93.65%	100.00%	96.72%	59
19	20	94.92%	94.92%	94.92%	59
20	21	93.55%	98.31%	95.87%	59
21	22	98.18%	91.53%	94.74%	59

Variables Terminal

7:22 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKI32koR5fBx8tuZH48gkrSLkRJ#scrollTo=8ZgZSFuZr_GD

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

21 22 98.18% 91.53% 94.74% 59

22 23 89.06% 95.00% 91.94% 60

23 24 95.08% 98.31% 96.67% 59

24 25 98.28% 95.00% 96.61% 60

25 26 96.67% 98.31% 97.48% 59

26 27 96.49% 94.83% 95.65% 58

27 28 95.08% 96.67% 95.87% 60

28 29 100.00% 100.00% 100.00% 58

29 30 100.00% 98.33% 99.16% 60

30 31 98.33% 100.00% 99.16% 59

31 32 94.92% 93.33% 94.12% 60

32 33 89.83% 89.83% 89.83% 59

33 34 100.00% 98.31% 99.15% 59

34 35 98.33% 100.00% 99.16% 59

35 36 100.00% 98.33% 99.16% 60

36 37 100.00% 96.61% 98.28% 59

Variables Terminal

7:22 PM Python 3

Gurmukhi Handwritten Character x

Ask Gemini

colab.research.google.com/drive/1TwnUyKI32koR5fBx8tuZH48gkrSLkRJ#scrollTo=hohUk4HosXec

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

37 38 96.36% 89.83% 92.98% 59

38 39 89.39% 100.00% 94.40% 59

39 40 96.43% 91.53% 93.91% 59

40 41 93.55% 98.31% 95.87% 59

CLASS-WISE F1-SCORE

[56] 0s plt.figure(figsize=(14, 6))

plt.bar(class_performance["Class"].astype(str), class_performance["F1-Score"])

plt.axhline(f1_tuned, linestyle="--", linewidth=2, label=f"Macro F1-Score: {f1_tuned:.2%}")

Variables Terminal

7:23 PM Python 3

colab.research.google.com/drive/1TwnUyKIi32koR5fbx8tuZH48gkrSLkRJ#scrollTo=hohUk4HosXec

Ask Gemini

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

[56] 0s

```
f1_tuned,
linestyle="--",
linewidth=2,
label=f"Macro F1-Score: {f1_tuned:.2%}"
)

plt.title(
    "F1-Score Across Gurmukhi Character Classes",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel("Character Class")
plt.ylabel("F1-Score")

plt.ylim(0, 1.05)

plt.legend()

plt.xticks(rotation=90)

plt.tight_layout()
plt.show()
```

F1-Score Across Gurmukhi Character Classes

Macro F1-Score: 96.53%

Variables Terminal

7:23 PM Python 3

colab.research.google.com/drive/1TwnUyKIi32koR5fbx8tuZH48gkrSLkRJ#scrollTo=hohUk4HosXec

Ask Gemini

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

[56] 0s

```
plt.show()
```

F1-Score Across Gurmukhi Character Classes

Macro F1-Score: 96.53%

Variables Terminal

7:23 PM Python 3

The figure is a bar chart titled "F1-Score Across Gurmukhi Character Classes". The y-axis is labeled "F1 Score" and ranges from 0.0 to 1.0. The x-axis is labeled "Character Class" and ranges from 1 to 41. Each bar represents the F1 score for a specific character class. A dashed horizontal line at the top of the chart indicates the "Macro F1-Score: 96.53%".

Character Class	F1 Score (approx.)
1	1.00
2	0.98
3	0.99
4	0.99
5	0.95
6	0.99
7	0.98
8	0.97
9	0.99
10	0.98
11	0.98
12	0.98
13	0.97
14	0.97
15	0.92
16	0.98
17	0.98
18	0.99
19	0.97
20	0.97
21	0.96
22	0.95
23	0.92
24	0.97
25	0.97
26	0.97
27	0.96
28	0.96
29	0.99
30	0.99
31	0.99
32	0.95
33	0.90
34	0.99
35	0.99
36	0.99
37	0.98
38	0.94
39	0.94
40	0.94
41	0.94

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

3	Testing Images	2426
4	Image Size	32 × 32
5	Baseline RF Accuracy	95.75%
6	HOG + RF Accuracy	78.52%
7	Tuned RF Accuracy	96.54%
8	Tuned RF Precision	96.65%
9	Tuned RF Recall	96.54%
10	Tuned RF F1-Score	96.53%

Next steps: [Generate code with final_summary](#) [New interactive sheet](#)

Variables Terminal 7:25 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

RAM Disk

SAVE TRAINED MODEL

```
[58] ✓ 7s
import joblib

model_filename = "gurmukhi_random_forest_model.pkl"

joblib.dump(
    rf_tuned,
    model_filename
)

print("Final model saved successfully.")
print("File:", model_filename)

... Final model saved successfully.
File: gurmukhi_random_forest_model.pkl

[59] ✓ 0s
from google.colab import files

files.download(
    "gurmukhi_random_forest_model.pkl"
)

...
```

Variables Terminal 7:28 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

SAVE CLASS LABELS

```
[50] import json

class_labels = [
    str(label)
    for label in rf_tuned.classes_
]

with open(
    "gurmukhi_class_labels.json",
    "w"
) as file:

    json.dump(
        class_labels,
        file,
        indent=4
    )

print("Class labels saved successfully.")
```

... Class labels saved successfully.

Variables Terminal 7:33 PM Python 3

Gurmukhi Handwritten Character Recognition using Random Forest

```
[50] print("Class labels saved successfully.")
Class labels saved successfully.

[51] files.download(
    "gurmukhi_class_labels.json"
)
```

Gurmukhi Handwritten Character

Ask Gemini

colab.research.google.com/drive/1TwnUyKI32koR5fBx8tuZH48gkrSLkRJ#scrollTo=7gdpKb3cuw1f

Gurmukhi Handwritten Character Recognition using Random Forest

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

CONCLUSION

This project developed a Random Forest-based handwritten Gurmukhi character recognition system using a publicly available labeled dataset containing 41 character classes.

The dataset was explored and preprocessed by converting the character images to grayscale, resizing them to 32×32 pixels, and normalizing the pixel values. Two feature representations were investigated: raw pixel features and Histogram of Oriented Gradients (HOG) features.

A baseline Random Forest model using raw pixel features achieved an accuracy of 95.75%. The HOG-based Random Forest achieved 78.52%, demonstrating that the selected feature representation had a significant effect on classification performance.

Hyperparameter optimization was then performed using RandomizedSearchCV with 3-fold cross-validation. The optimized Random Forest achieved a test accuracy of 96.54%, with macro precision of 96.65%, macro recall of 96.54%, and macro F1-score of 96.53%.

The model was further evaluated using a classification report, confusion matrix, class-wise performance analysis, confidence analysis, and visualization of misclassified characters. An unseen-image prediction module was also developed, allowing a new handwritten character image to be uploaded and classified by the trained model.

The final system demonstrates that Random Forest combined with suitable image preprocessing and feature representation can effectively recognize handwritten Gurmukhi characters.

Variables Terminal

7:33 PM Python 3